الجمهورية الجزائرية الديمقراطية الشعبية

وزارة التعليم العالي و البحث العلمي

People's Democratic Republic Of Algeria

Ministry of Higher Education and Scientific Research

MINI PROJECT REPORT

Agricultural Production Management System

(APMS)

Group Members

KHEFFACHE Semhane (**Leader**)

HATTABI Hadil

SOUAK Maroua

MAHFOUDIA Nour el houda Imene

December,22nd ,2023.

Outline:

- **Introduction** 3
- **Explanation of the problem treated** 3
- **About Data** 3
- **Explanation of the design choices for the solution** 4
- **Solution Description** 5

  Part one

  Part two

  How do we perform the preliminaries ?

  How can we find the winner in any Area?

  How can we find the winner in the City or Wilaya or Country?

  Illustrations

- **Running time (BST/AVL)** 12
- **Conclusion**
- **Appendix** 13

  References

  Who did What?

➢ Introduction:

To promote the responsible use of water and energy in agriculture, the government has implemented a program. This initiative offers monthly incentives across various categories including Vegetables, Fruits, Cereals, Olives and Dates for farmers who display exemplary performance based on their sales figures as well as electricity/water consumption patterns and pesticide utilization levels.The government has enlisted experts to oversee farmers and ensure that pesticides are being used responsibly in this program. Any excessive use of pesticides can result in penalties for the farmers involved. Additionally, by recognizing resource-efficient production methods through local and national selection processes, participating farmers who practice sustainable farming will be rewarded with hopes of promoting wider adoption of these important principles.Not only does this approach acknowledge diligent farmers, but it also supports larger objectives regarding protecting the environment and promoting sustainable agriculture - both vital for continued success in the years to come.

➢ Explanation of the problem treated:

The project seeks to develop an **Agricultural Production Management System** (APMS) with comprehensive coordination capabilities for various agriculture-related aspects. The system will feature the following components:

Farmers have the opportunity to produce an account and provide monthly details regarding their products, including sales figures as well as water usage.

The Ministry plans on conducting pesticide level tests in products produced by farmers. If necessary, the ministry will impose penalties for those that fail to meet specific standards. Additionally, experts will inspect these products before they are sold to ensure their quality and safety for consumers.

The system will assess farmers' inputted data and the penalties enforced by the Ministry, with a view to identifying winners across various tiers. Winners will be recognized on a local, regional (city or Wilaya), and national level. Furthermore, the platform shall monitor monthly champions for each product category.Farmers' information will be stored in a manner that simplifies search and access within the system. Furthermore, it enables the Ministry to review sales of any farmer during any month as well as all farmers at every level promptly. Additionally, details concerning winners of prize categories and monthly penalties imposed on farmers are available for scrutiny by authorities.

Please refer to the figure (*) in the Appendix for improved understanding.

➢ About Data:

The following data has been selected in CSV format:

- FarmerInformation: includes FarmerID, LastName, Age, Gender, LandID, Area, City, and Wilaya.

- MonthlyReport: includes FarmerID, NumHelpers, ProducesFruits, ProducesVegetables, ProducesCereals, ProducesDates, ProducesOlives, FruitsSales, VegetablesSales, CerealsSales, DatesSales, OlivesSales, WaterConsumption, ElectricityConsumption, Month, and Year.

- FarmerLocation: includes Area, City, Wilaya, and wilaya_code.

➢ Explanation of the design choices for the solution:

To address our requirements and resolve the issue through innovation, we've designed a range of classes equipped with appropriate ADTs. can be explained as following:

The ADT	The problem	Why
Stack	Comparing between farmers and get the winner at any level	Firstly, we initiate a stack containing one farmer. Subsequently, our process entails consistent examination of each new farmer against the topmost individual on the existing stack. Whenever there is an identification of a fresh victor over the previous holder, we eject that older winner and insert this newly crowned adept onto our stack instead. In instances where several farmers attain victorious status simultaneously under certain criteria for comparison, these champions will be reassessed using alternative parameters before reinstatement to their rank in accordance with initial metrics observed; all through stacking processes paramount due to O(1) time complexity which benefits from stacks' LIFO (Last-In-First-Out) property thus facilitating hassle-free addition or removal operations concerning winners kept track within them.

Vector	Storing Areas in City, Storing Cities in Wilaya Storing Wilayas in Country	For instance, in a city with multiple areas, if we want to find out about all the farmers in that city, we need to go through each area and collect the information about their farmers. Thus, in all instances of searching, displaying or finding a winner, it is necessary to traverse through all areas. This process takes linear time and with all Abstract Data Types (ADTs), we have opted for vectors to simplify the process and avoid convoluted code (preventing spaghetti code). The same applies to other scenarios.
BST and AVL tree	Storing all farmers information	We choose the farmer to be the main object rather than the product so we store the farmers into the BST with all their information either constant information (FarmerID,LastName,Age,Gender,LandID,Area,City,Wilaya.) or the monthly information(sales, water, electricity, …etc) and using the ID as constraint of the BST (before inserting we need to compare farmers with their ID) and we construct 5 BST for each product (category) to easily find the winner of the AREA , CITY, WILAYA and COUNTRY for each category at a given month.

- **<u>Solution Description:</u>**

In order to handle both monthly and general farmer data efficiently, the solution was divided into two parts. The first part involved a Ministry interface that allowed for all necessary operations such as searching winners or retrieving penalties specific to the ministry's needs. Meanwhile, the second sector replicated the Farmer Interface which solely focused on managing data related to farmers.

★ Part one:

We have defined four classes as depicted in Figure 2 on page 8.These include:

- **Area**: a class that helps find the winner in a given category within a specific area.

- **City**: a class that helps find the winner in a given category within a specific city. Each city has a vector of its respective areas.

- **Wilaya**: a class that helps find the winner in a given category within a specific Wilaya. Each Wilaya has a vector of its respective cities.

- **Country**: a class that helps find the prize winner in a given category across the entire country. We chose to define these four classes to facilitate finding winners, getting all penalties, getting all farmers, and more, in a given area, city, Wilaya, or the entire Country.

★ Part two:

Figure 2 on page 8 illustrates the two classes we created.

One class is named "**Farmer**" and houses constant farmer information, like ID, name, age, gender and more. The second class called "**Monthly Report**," stores dynamic data including water consumption, electricity usage figures as well as sales records etcetera. Both these classes integrate smoothly to execute any required actions

★ How do we perform the preliminaries ?

The qualifications for the competition are divided into four stages. The first stage involves comparing all farmers in a specific region, and the winner is selected for each Area. This concludes the first stage, and the winners from each Area advance to the second stage.

The second stage involves comparing the winners from different Areas within the same

City. Consequently, all winners at the City level are determined, concluding the quarter-finals with the qualification of only the winning farmers at the City level. These City-level winners are then compared to extract the winners at the state level, concluding the semi-finals. The final stage involves comparing the state-level winners to extract one overall winner at the national level. This individual is declared the winner of the category and the recipient of the prize. (as shown in Figure1 page 9).

★ How can we find the winner in any Area ?

we can find the winner in a given Area following these steps :

<u>Get the farmer from the Binary Search Tree:</u>

Using the Binary Search Tree in a specific category we can get information about farmers in each Area.

<u>Comparison using Stack ADT:</u>

We create a Stack and initialize it with the first farmer extracted from the Binary Search Tree and this to facilitate comparisons between farmers in the same Area using the amount of sales, water consumption.

now how we use the stack properly to get one winner farmer among all the Area's farmers.when we get a new farmer from the BST, compare them with the farmer at the top of the stack.if the sales are high and water consumption is low for the new farmer we pop the top of the stack and push the new farmer onto the stack.

if the new farmer has low sales and high water consumption, we skip the next new farmer, and if the sales and water consumption are equal for both new farmer and the top of the stack, we need to re-compare them according to the electricity consumption and pesticides level(as an exception handling)

We need to redo this procedure until the top of the stack is the farmer with the high amount of sales after penalties and lower water and electricity consumption.

In this process the stack is pretended to compare and update the current winner based on the comparison conditions we made .

(as shown in Figure 1.1 page 9).

★ How can we find the winner in the City or Wilaya or Country?

The method for determining the winner can be applied at the City, Wilaya, or Country level as they all contain a vector of their corresponding entities. Specifically, Cities have a vector of Areas, Wilayas have a vector of Cities and Countries possess a vector of Wilayas.

To illustrate, the process of determining the City's champion involves starting with adding the initial winnerArea farmer to a stack from our vector. Subsequently, we would compare this victor against other champions in each Area using an identical approach utilized for finding winners within each area individually. Ultimately, we keep track and select the ultimate champion of all areas as determined by these comparisons.

(as shown in Figure 1.2 page 10).

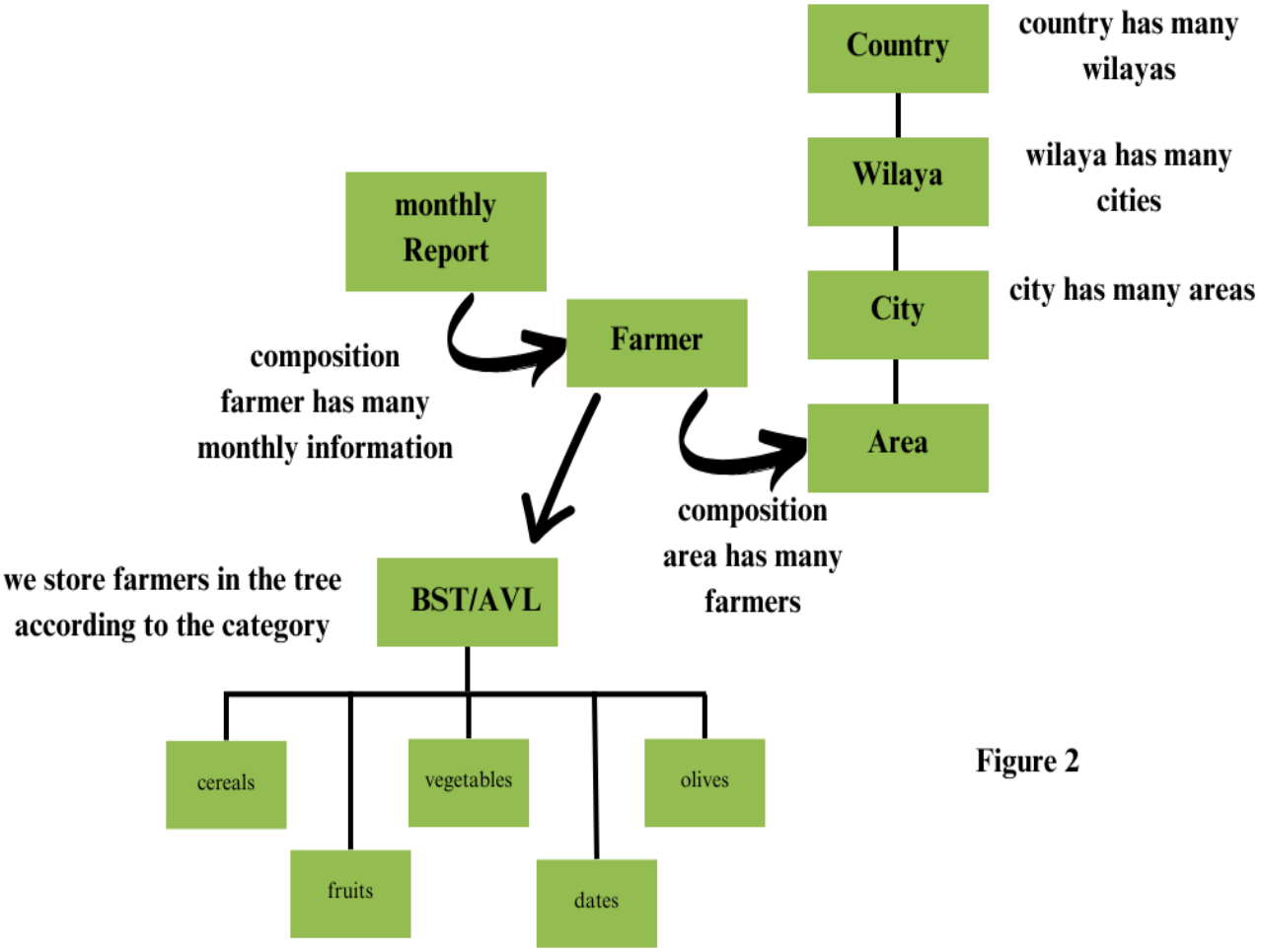

Figure 2

This project involves the necessary steps to select a winner in a given category:

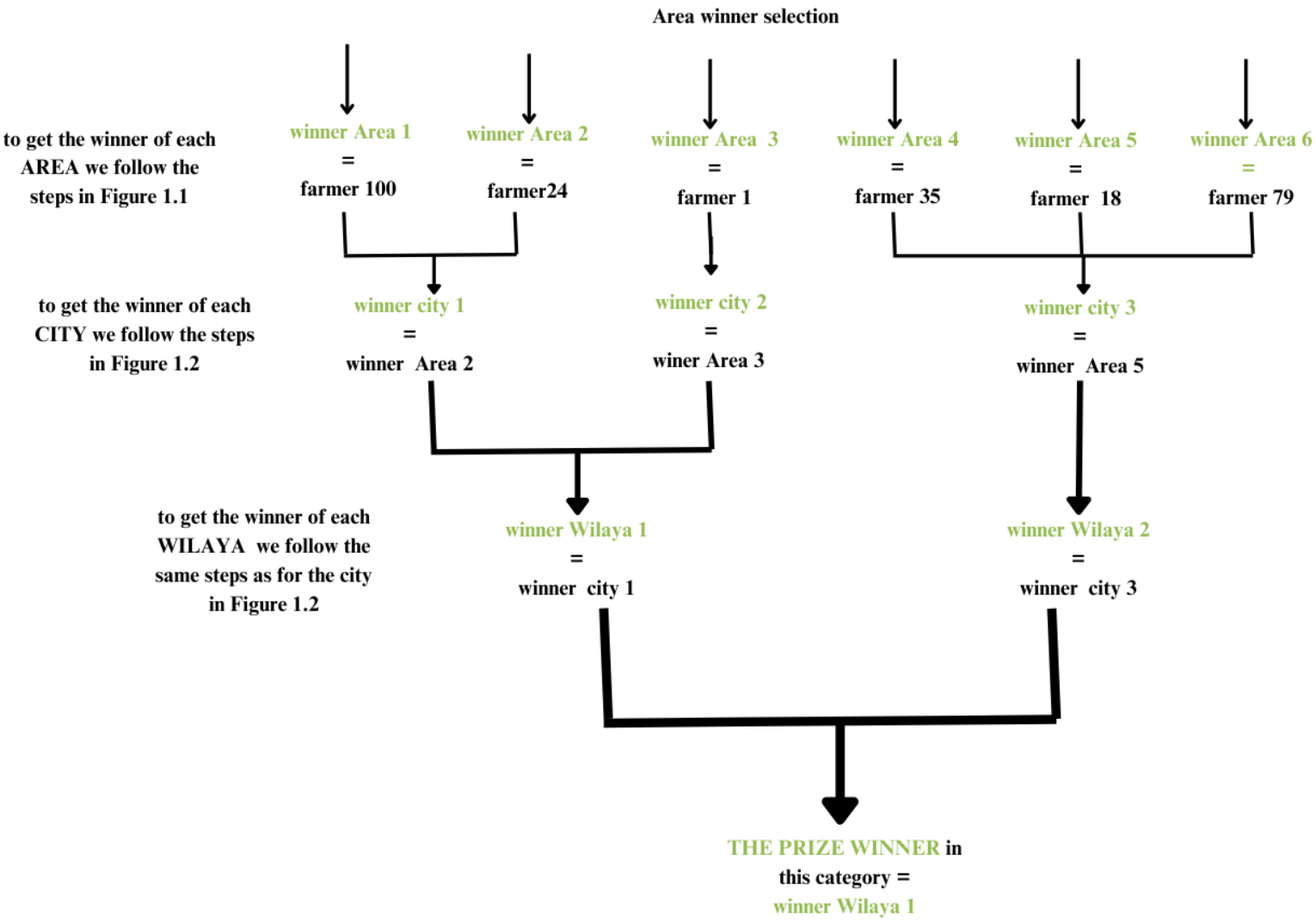

Figure 1 (explanation)

This is an illustration of how we select the winner in the Area

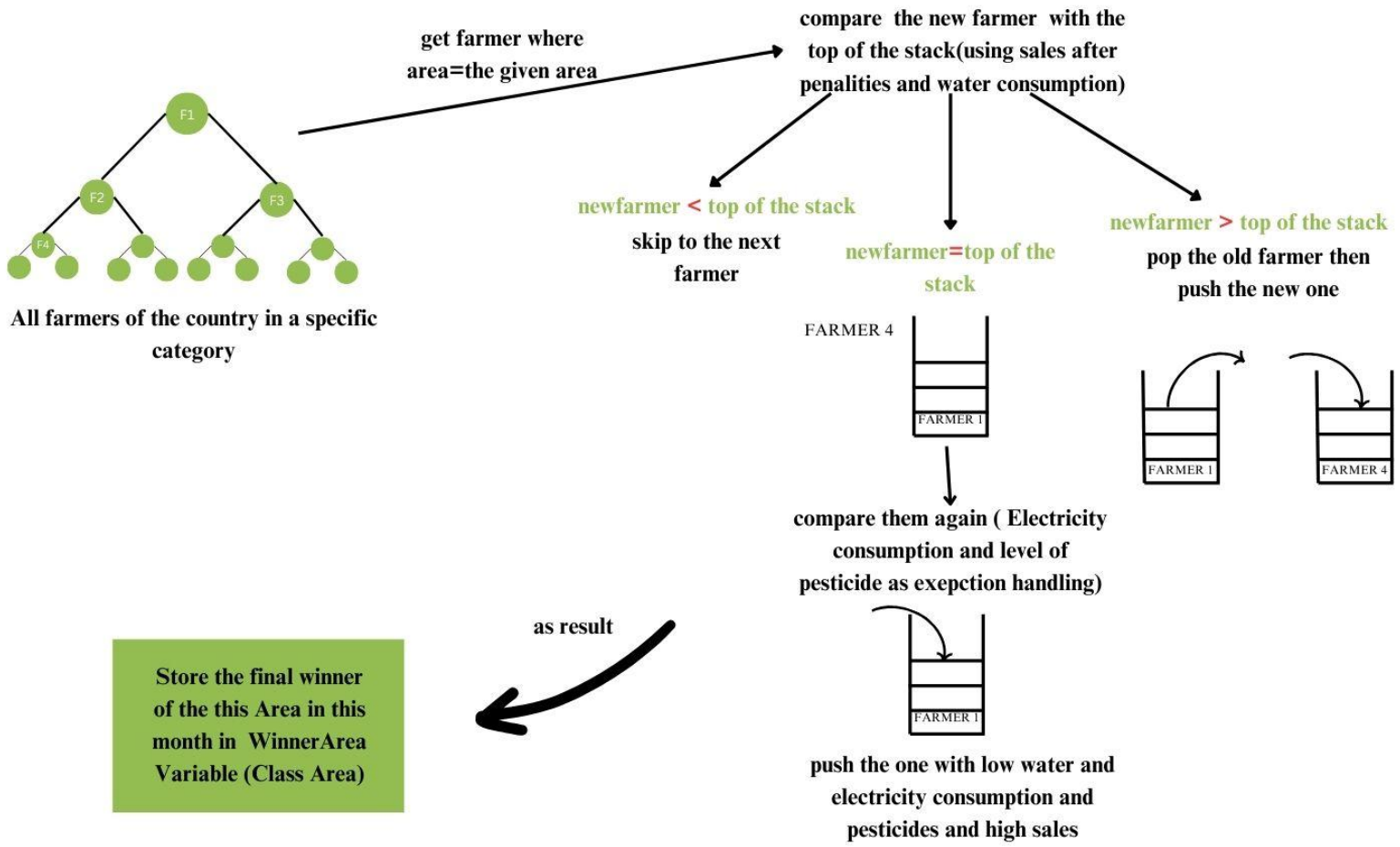

Figure 1.1

This is how we determine the winner in city/wilaya/ country

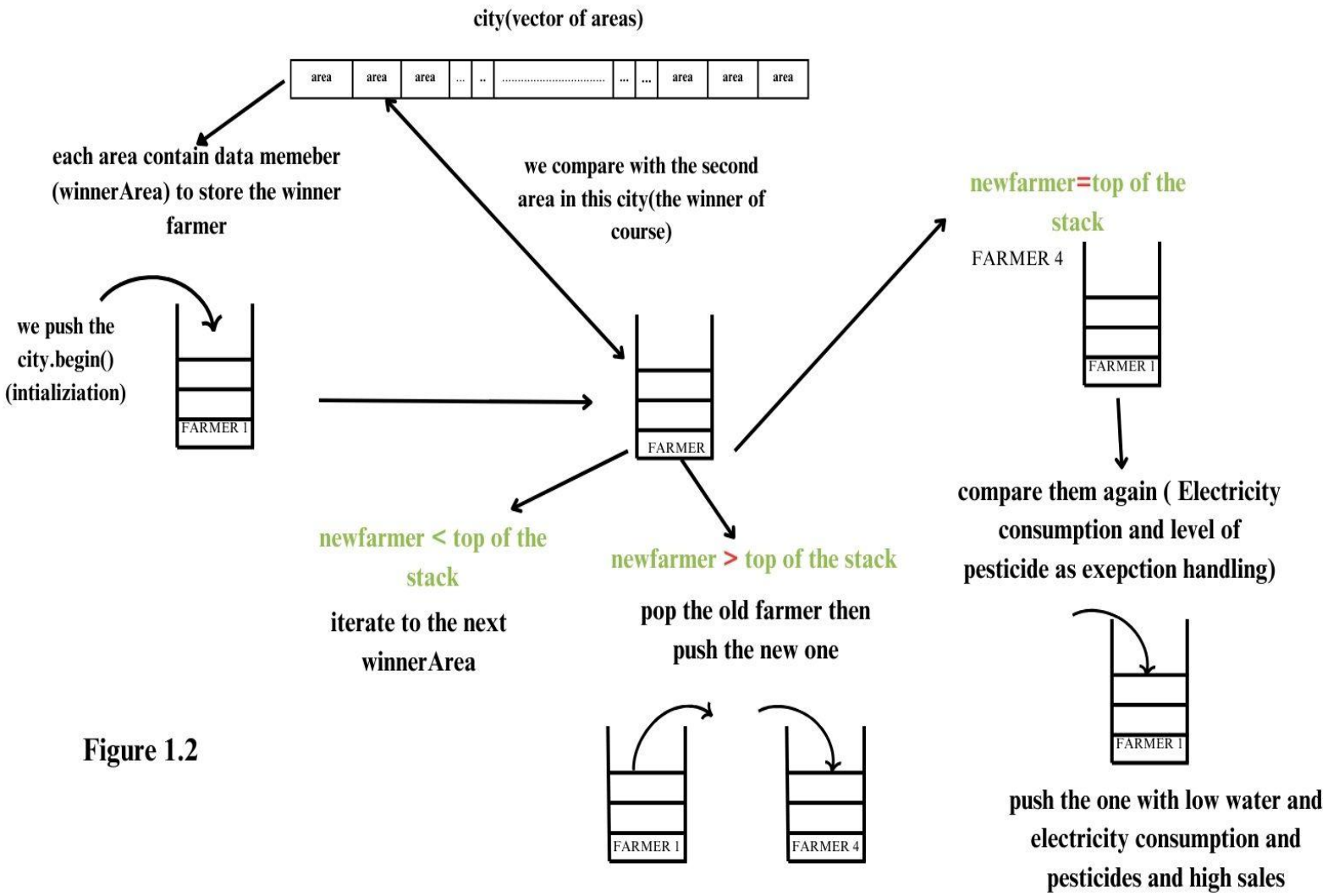

Figure 1.2

➢ Running time BST/AVL:

Various amounts of data were used to test the program with both BST and AVL trees.

After running the program using this data we get the following results:

Part	BST(binary search tree)	AVL tree
Running time	25433 milliseconds	15278 milliseconds

➢ Conclusion :

By examining the runtime of BST and AVL, we can ascertain which one exhibits greater efficiency after analyzing the average execution time shown in the previous table:

In summary, when we applied the running time calculations on both part I and part II of the project we saw that the running time in AVL tree is much less than in BST trees , and here we deduce that AVL trees represents a better choice if we want to ensure consistent performance

Appendix:

References:

- Implementation of the BST from the textbook..
- The data file generated using AI (to fit our needs exactly).
- Various AI tools for debugging.

Who did What?

Right from the start, we held meetings every day and conversed about everything collectively. In general, mutual assistance was extended amongst us all:

KHEFFACHE Semhane:

-Avl tree implementation.(1/2)

-BST implementation .(1/2)

-Generating FarmerInformation.csv file.

-Generating FarmerLocation.csv file.

-Generating MonthlyReport.csv file.

-Report's introduction and idea figure(*).

-Testing & debugging (1/4)

-Conclusion (1/4)

HATTABI Hadil:

-Implementation of classes Area City Wilaya and Country (1/2)

-Implementation of main. (1/2).

-Testing & debugging (1/4)

-Report.

-All graphical representations.

-Conclusion (1/4)

SOUAK Maroua:

-Implementation of classes Area City Wilaya and Country (1/2)

-Implementation of main. (1/2)

-Testing & debugging. (1/4)

-Checking function for names Area and City and Wilaya in files.

-Program of testing Running time.

-Conclusion (1/4)

MAHFOUDIA Nour el houda Imene:

-Avl tree implementation.(1/2)

-BST implementation .(1/2)

-Testing & debugging. (1/4)

-Project linking.

-Printing functions from files.

-Reader functions for files.

-Farmer and MonthlyReport implementation.

-Conclusion (1/4)

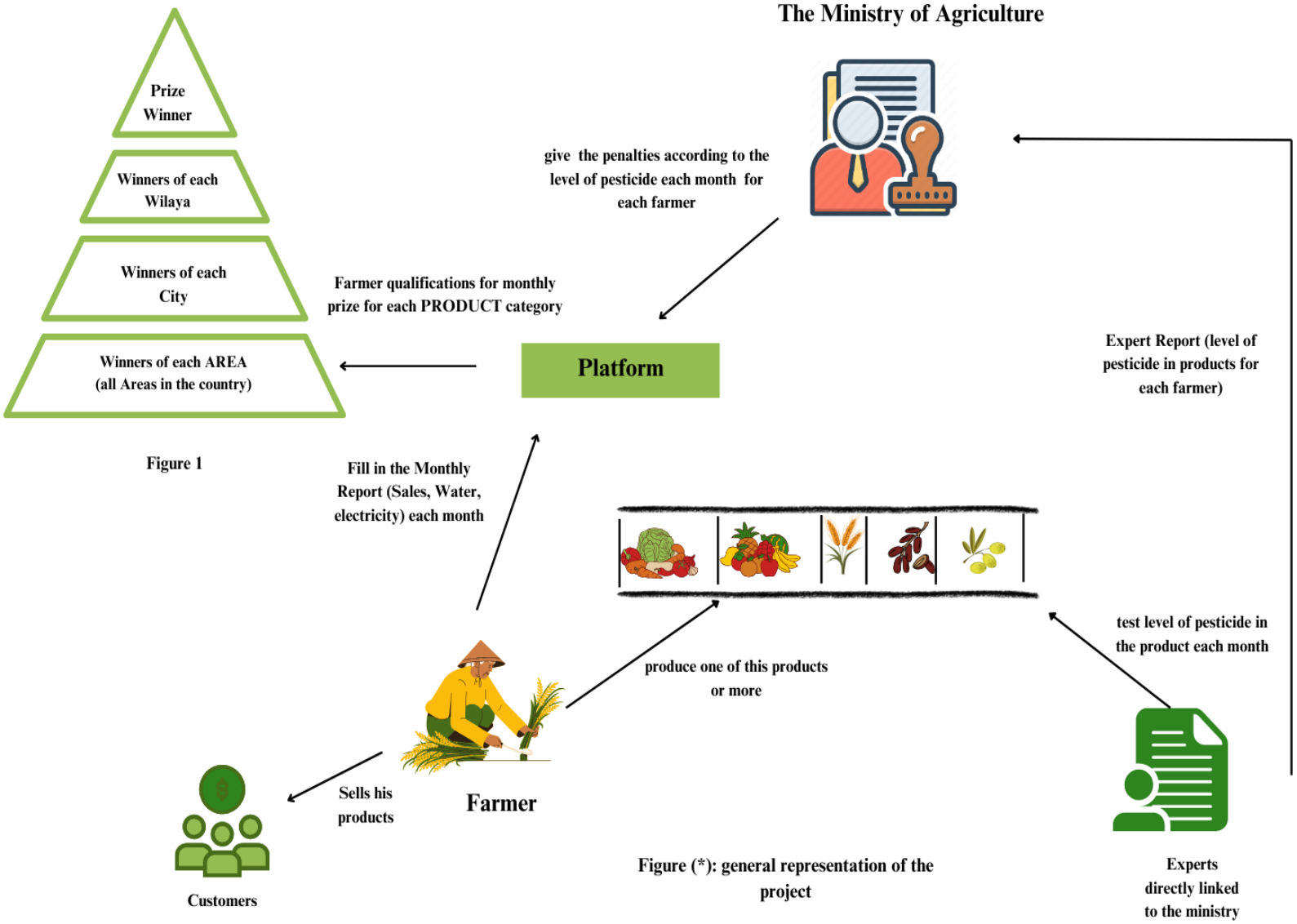

Figure (*): general representation of the project

```
Enter target year: 2022
Enter target month: 1
-------------------WELCOME TO THE AGRICULTURAL PRODUCTION MANAGEMENT SYSTEM-------------------

1.Home page
2.Farmer Interface
3.Admin page(APMS)
Please choose one page!
```

Test cases :

1-Home page:

```
Please choose one page!
1
-----Home page-----
Enter the category you wish to know its winner (cereals/fruits/vegetables/dates/olives): olives
the winner of the country is:
Country Winner: 237 Bensaoula


1.continue
2.log out
```

2-Farmer Interface:

```
Please choose one page!
2
-----Farmer Interface-----
WELCOME dear Farmer
1.Login(I have already an account)
2.Sign up( don't have an account)
Please, choose one page!
1
a.My profile
b.Update my additional Data
Please, choose one action!
a
My profile
enter your id
201
Information of farmer from ElKhroub, Constantine wilaya of Constantine
Name: Abdi
National ID: 201
Age: 35
Gender: Male
-------------------------------------------------------------------------------------------
would you want to update your additional information(Y/y->yes)or(N/n->no)
y
Update my additional Data
updating sales
1
updating consumption
Enter this month's water consumption(in cubic meter): 5000
Enter this month's electricity(in kilo watt): 2000
updating productSales
Enter the number of products sold at your farm (1-5): 2
Enter product name (Cereals, Vegetables, Fruits, Dates, Olives): fruits
Enter month and year of sales:
Enter sales of fruits in Algerian Dinar (DZDA) :10000
Enter product name (Cereals, Vegetables, Fruits, Dates, Olives): dates
Enter month and year of sales:
Enter sales of dates in Algerian Dinar (DZDA) :777777
-------------------------------------------------------------------------------------------
would you want to update your additional information(Y/y->yes)or(N/n->no)
```

3-Admin page(APMS):

Winner are for exemple

```
-----Admin page(APMS)-----
enter month: 1
enter year: 2022
WELCOME, to the dashbord
1.See all farmer's accounts
2.Check on all sales of all farmers
3.See winners in the given(Area/ City/ Wilaya)
4.Penalities
5.Prize winner (Country winner)
6.Search of a specific farmer
Enter an option : 3
Enter the category of the winner you want to display [Cereals, Vegetables, Fruits, Dates, Olives] :
fruits
enter the location that you are looking for its winner, Area  or  City or Wilaya
1.Area
2.City
3.Wilaya
1
enter the area you want to display its winner
ElKhroub
Winner of ElKhroub is: Farmer 213 Louafi
```

The city winner

```
WELCOME, to the dashbord
1.See all farmer's accounts
2.Check on all sales of all farmers
3.See winners in the given(Area/ City/ Wilaya)
4.Penalities
5.Prize winner (Country winner)
6.Search of a specific farmer
Enter an option : 3
Enter the category of the winner you want to display [Cereals, Vegetables, Fruits, Dates, Olives] :
dates
enter the location that you are looking for its winner, Area  or  City or Wilaya
1.Area
2.City
3.Wilaya
2
enter the city you want to display its winner
Algiers
Winner of Algiers is: Farmer 295 Kerbouche
```

The prize winner

```
WELCOME, to the dashbord
1.See all farmer's accounts
2.Check on all sales of all farmers
3.See winners in the given(Area/ City/ Wilaya)
4.Penalities
5.Prize winner (Country winner)
6.Search of a specific farmer
Enter an option : 5
Enter the category of the winner you want to display [Cereals, Vegetables, Fruits, Dates, Olives] :
olives
Country Winner: 237 Bensaoula
```

Penalties (city)

```
WELCOME, to the dashbord
1.See all farmer's accounts
2.Check on all sales of all farmers
3.See winners in the given(Area/ City/ Wilaya)
4.Penalities
5.Prize winner (Country winner)
6.Search of a specific farmer
Enter an option : 4
Enter the category of the farmers you want to display [Cereals, Vegetables, Fruits, Dates, Olives] :
dates
enter the location that you are looking for (Area/City/Wilaya):
1.Area
2.City
3.Wilaya
4.Country
2
enter the city you want to display its record of penalities:
TiziOuzou
---------List of pinalities of TiziOuzou :----------
Farmer in Area: Azeffoun
Information of farmer from Azeffoun, TiziOuzou wilaya of TiziOuzou
Name: Belkalem
National ID: 263
Age: 42
Gender: Male
ID of the land: 663
Level of pesticides applied: 1
Farmer in Area: TiziRached
Information of farmer from TiziRached, TiziOuzou wilaya of TiziOuzou
Name: Touati
National ID: 209
Age: 36
Gender: Male
ID of the land: 609
Level of pesticides applied: 3
Farmer in Area: Azeffoun
Information of farmer from Azeffoun, TiziOuzou wilaya of TiziOuzou
Name: Belkalem
National ID: 263
Age: 42
Gender: Male
ID of the land: 663
Level of pesticides applied: 0
Farmer in Area: TiziRached
Information of farmer from TiziRached, TiziOuzou wilaya of TiziOuzou
Name: Touati
National ID: 209
Age: 36
Gender: Male
ID of the land: 609
Level of pesticides applied: 3
Farmer in Area: Azeffoun
Information of farmer from Azeffoun, TiziOuzou wilaya of TiziOuzou
Name: Belkalem
National ID: 263
Age: 42
Gender: Male
ID of the land: 663
Level of pesticides applied: 1
```

Penalties (area)

```
WELCOME, to the dashbord
1.See all farmer's accounts
2.Check on all sales of all farmers
3.See winners in the given(Area/ City/ Wilaya)
4.Penalities
5.Prize winner (Country winner)
6.Search of a specific farmer
Enter an option : 4
Enter the category of the farmers you want to display [Cereals, Vegetables, Fruits, Dates, Olives] :
cereals
enter the location that you are looking for (Area/City/Wilaya):
1.Area
2.City
3.Wilaya
4.Country
1
enter the area you want to display its record of penalities:
Berrahal
---------List of pinalities of Berrahal :----------
Farmer in Area: Berrahal
Information of farmer from Berrahal, Annaba wilaya of Annaba
Name: Belaid
National ID: 286
Age: 44
Gender: Female
ID of the land: 686
Level of pesticides applied: 2
```

Search farmer with id

```
-------------------WELCOME TO THE AGRICULTURAL PRODUCTION MANAGEMENT SYSTEM-------------------

1.Home page
2.Farmer Interface
3.Admin page(APMS)
Please choose one page!
3
-----Admin page(APMS)-----
enter month: 1
enter year: 2022
WELCOME, to the dashbord
1.See all farmer's accounts
2.Check on all sales of all farmers
3.See winners in the given(Area/ City/ Wilaya)
4.Penalities
5.Prize winner (Country winner)
6.Search of a specific farmer
Enter an option : 6
Enter farmer ID to search: 201
ID: 201
Abdi
35
Male
601
Constantine
Constantine
ElKhroub
1.continue searching in the farmer page
2.exit from the farmer page
```

Running time:

```
1.continue searching in the farmer page
2.exit from the farmer page
2


1.continue
2.log out

2
Program execution time: 475047 milliseconds
```

★ CODE :

The binary tree:

```
            root = clone(rhs.root);
        }
        return *this;
    }

    BinarySearchTree *GetLeft()
    {
        root->GetLeftNode();
    }

    BinarySearchTree *GetRight()
    {
        root->GetRightNode();
    }

    BinaryNode *GetRoot()
    {
        return root;
    }

    Farmer GetFarmer()
    {
        root->GetFarmerNode();
    }

    // function to print id and name of farmer of a specific area
    void PrintFarmerByArea(string areaName, int year, int month)
    {
        printFarmersArea(this->GetRootNode(), areaName, year, month);
    }

    // function to print id and name of farmer of a specific city
    void PrintFarmerByCity(string cityName, int year, int month)
    {
        printFarmersCity(this->GetRootNode(), cityName, year, month);
    }

    // function to print id and name of farmer of a specific wilaya
    void PrintFarmerByWilaya(string wilayaName, int year, int month)
    {
        printFarmersWilaya(this->GetRootNode(), wilayaName, year, month);
    }

    // function that inserts a farmer into the binary search tree according to the product produced
    // use a random tree in main to call this function on the farmer to store him in corresponding tree
    void InsertFarmer(Farmer &obj);

    // function that returns root of tree according to the product made by the farmer
    BinarySearchTree *GetProductTreeRoot(const string &product);

private:
    BinaryNode *root;

    BinaryNode *GetRootNode()
    {
        return root;
    }

    // Private member functions for BinarySearchTree
    void insert(Farmer &x, BinaryNode *&t)
    {
        if (t == nullptr)
            t = new BinaryNode{x, nullptr, nullptr};
        else if (x.GetId() < t->element.GetId())
            insert(x, t->left);
        else if (t->element.GetId() < x.GetId())
            insert(x, t->right);
        else
            return;
    }
```

```
        bool contains(Farmer &x, BinaryNode *&t)
        {
            if (t == nullptr)
                return false;
            else if (x.GetId() < this->element.GetId())
                return contains(x, t->left);
            else if (this->element.GetId() < x.GetId())
                return contains(x, t->right);
            else
                return true;
        }

        void makeEmpty(BinaryNode *&t)
        {
            if (t != nullptr)
            {
                makeEmpty(t->left);
                makeEmpty(t->right);
                delete t;
            }
            t = nullptr;
        }

        void printTree(BinaryNode *t, ostream &out) const
        {
            if (t != nullptr)
            {
                printTree(t->left, out);
                out << t->element.GetId();
                out << endl;
                printTree(t->right, out);
            }
        }

        BinaryNode *clone(BinaryNode *t) const
        {
            if (t == nullptr)
                return nullptr;
            else
                return new BinaryNode{t->element, clone(t->left), clone(t->right)};
        }

        BinaryNode *findMin(BinaryNode *t) const
        {
            if (t == nullptr)
                return nullptr;
            while (t->left != nullptr)
                t = t->left;
            return t;
        }
    };

    // Copy constructor
    BinarySearchTree(const BinarySearchTree &rhs) : root{nullptr}
    {
        root = clone(rhs.root);
    }

    // Destructor
    virtual ~BinarySearchTree()
    {
        makeEmpty(root);
    }

    // Public member functions
    bool contains(Farmer &x) const
    {
        return contains(x, root);
    }

    bool isEmpty() const
    {
        return root == nullptr;
    }

    void printTree() const
    {
        printTree(root, std::cout);
    }
```

```cpp
void remove(Farmer &x, BinaryNode *&t)
{
    if (t == nullptr)
        return;
    if (x.GetId() < t->element.GetId())
        remove(x, t->left);
    else if (t->element.GetId() < x.GetId())
        remove(x, t->right);
    else if (t->left != nullptr && t->right != nullptr) // Two children
    {
        t->element = findMin(t->right)->element;
        remove(t->element, t->right);
    }
    else
    {
        BinaryNode *oldNode = t;
        t = (t->left != nullptr) ? t->left : t->right;
        delete oldNode;
    }
}

bool contains(Farmer &x, BinaryNode *t) const
{
    if (t == nullptr)
        return false;
    else if (x.GetId() < t->element.GetId())
        return contains(x, t->left);
    else if (t->element.GetId() < x.GetId())
        return contains(x, t->right);
    else
        return true;
}

void makeEmpty(BinaryNode *&t)
{
    if (t != nullptr)
    {
        makeEmpty(t->left);
        makeEmpty(t->right);
        delete t;
    }
    t = nullptr;
}

void printTree(BinaryNode *t, std::ostream &out) const
{
    if (t != nullptr)
    {
        printTree(t->left, out);
        out << t->element.GetId();
        out << endl;
        printTree(t->right, out);
    }
}

// utility function to print all farmers in a specific area
// handles area repitition by precising which city we are in
void printFarmersArea(BinaryNode *t, string areaName, int year, int month)
{
    if (t != nullptr)
    {
        printFarmersArea(t->left, areaName, year, month);
        if (t->element.GetArea() == areaName && t->element.GetMonthlyStat().GetMonth() == month && t->element.GetMonthlyStat().GetYear() == year)
        {
            cout << "Farmer " << t->element.GetId() << ": " << t->element.GetName() << ", " << t->element.GetArea() << ", " << t->element.GetCity() << endl;
        }
        printFarmersArea(t->right, areaName, year, month);
    }
}

// utility function to print all farmers in a specific city
// handles area repitition by precising which wilaya we are in
void printFarmersCity(BinaryNode *t, string cityName, int year, int month)
{
    if (t != nullptr)
    {
        printFarmersCity(t->left, cityName, year, month);
        if (t->element.GetCity() == cityName && t->element.GetMonthlyStat().GetMonth() == month && t->element.GetMonthlyStat().GetYear() == year)
        {
            cout << "Farmer " << t->element.GetId() << ": " << t->element.GetName() << ", " << t->element.GetCity() << ", " << t->element.GetWilaya() << endl;
        }
        printFarmersCity(t->right, cityName, year, month);
    }
}
```

BinarySearchTree.cpp

```cpp
#include <iostream>
#include "BinarySearchTree.hpp"

using namespace std;

// initializing 5 binary search trees each correspond to a product and each store farmers
BinarySearchTree *Cereals = new BinarySearchTree();
BinarySearchTree *Vegetables = new BinarySearchTree();
BinarySearchTree *Fruits = new BinarySearchTree();
BinarySearchTree *Dates = new BinarySearchTree();
BinarySearchTree *Olives = new BinarySearchTree();

Comment Code
void BinarySearchTree::InsertFarmer(Farmer &obj)
{
    for (int i = 0; i < obj.GetMonthlyStat().GetNumOfProducts(); i++)
    {
        if (obj.GetMonthlyStat().GetProductName() == "cereals" || obj.GetMonthlyStat().GetProductName() == "Cereals")
        {
            Cereals->insert(obj);
            cout << "cereals tree: ";
            Cereals->printTree();
            cout << endl;
        }
        if (obj.GetMonthlyStat().GetProductName() == "dates" || obj.GetMonthlyStat().GetProductName() == "Dates")
        {
            Dates->insert(obj);
            cout << "dates tree: ";
            Dates->printTree();
            cout << endl;
        }
        if (obj.GetMonthlyStat().GetProductName() == "olives" || obj.GetMonthlyStat().GetProductName() == "Olives")
        {
            Olives->insert(obj);
            cout << "olives tree: ";
            Olives->printTree();
            cout << endl;
        }
        if (obj.GetMonthlyStat().GetProductName() == "vegetables" || obj.GetMonthlyStat().GetProductName() == "Vegetables")
        {
            Vegetables->insert(obj);
            cout << "vegetables tree: ";
            Vegetables->printTree();
            cout << endl;
        }
        if (obj.GetMonthlyStat().GetProductName() == "fruits" || obj.GetMonthlyStat().GetProductName() == "Fruits")
        {
            Fruits->insert(obj);
            cout << "fruits tree: ";
            Fruits->printTree();
            cout << endl;
        }
    }
}

Comment Code
BinarySearchTree *BinarySearchTree::GetProductTreeRoot(const string &product)
{
    string lowercaseProduct = product; // Make a copy to avoid modifying the original string
    transform(lowercaseProduct.begin(), lowercaseProduct.end(), lowercaseProduct.begin(), ::tolower);

    if (lowercaseProduct == "cereals")
        return Cereals;
    else if (lowercaseProduct == "vegetables")
        return Vegetables;
    else if (lowercaseProduct == "fruits")
        return Fruits;
    else if (lowercaseProduct == "dates")
        return Dates;
    else if (lowercaseProduct == "olives")
        return Olives;
    else
        return nullptr; // Return nullptr for unknown products
}
```

Avl tree:

AVLTree.hpp > ...

Click here to ask Blackbox to help you code faster

```cpp
#pragma once
#include <iostream>
#include <algorithm>
#include <cctype>
#include "Farmer.hpp"
using namespace std;

class AvlTree
{
public:
    AvlTree() : root{nullptr} {}

    // Copy constructor
    AvlTree(const AvlTree &rhs) : root{nullptr}
    {
        root = clone(rhs.root);
    }

    // Destructor
    virtual ~AvlTree()
    {
        makeEmpty(root);
    }

    // Public member functions
    bool contains(Farmer &x) const
    {
        return contains(x, root);
    }

    bool isEmpty() const
    {
        return root == nullptr;
    }

    void printTree() const
    {
        printTree(root, std::cout);
    }

    void insert(Farmer &x)
    {
        insert(x, root);
    }

    void remove(Farmer &x)
    {
        remove(x, root);
    }

    AvlTree &operator=(const AvlTree &rhs)
    {
        if (this != &rhs)
```

```
AvlTree &operator=(const AvlTree &rhs)
{
    if (this != &rhs)
    {
        makeEmpty(root);
        root = clone(rhs.root);
    }
    return *this;
}
// dunno if we remove it
AvlTree *GetLeft()
{
    root->GetLeftNode();
}

AvlTree *GetRight()
{
    root->GetRightNode();
}

Farmer GetFarmer()
{
    root->GetFarmerNode();
}

// function that inserts a farmer into the AVL tree according to the product produced
// use a random tree in main to call this function on the farmer to store him in corresponding tree
void InsertFarmer(Farmer &obj);

// function that returns root of tree according to the product made by the farmer
AvlTree *GetProductTreeRoot(const string &product);

struct AvlNode
{
    AvlNode *left;
    AvlNode *right;
    Farmer element;
    int height;

    AvlNode(Farmer &TheFarmer, AvlNode *lt, AvlNode *rt, int h = 0) : element(TheFarmer), left(lt), right(rt), height(h) {}

    // Private member functions for AvlNode
    AvlNode *GetLeftNode()
    {
        return left;
    }
    AvlNode *GetRightNode()
    {
        return right;
    }
    Farmer GetFarmerNode()
    {
        return element;
    }

    void insert(Farmer &x, AvlNode *&t)
    {
```

```
AvlTree *GetProductTreeRoot(const string &product);

struct AvlNode
{
    AvlNode *left;
    AvlNode *right;
    Farmer element;
    int height;

    AvlNode(Farmer &TheFarmer, AvlNode *lt, AvlNode *rt, int h = 0) : element(TheFarmer), left(lt), right(rt), height(h) {}

    // Private member functions for AvlNode
    AvlNode *GetLeftNode()
    {
        return left;
    }
    AvlNode *GetRightNode()
    {
        return right;
    }
    Farmer GetFarmerNode()
    {
        return element;
    }

    void insert(Farmer &x, AvlNode *&t)
    {
        if (t == nullptr)
        {
            t = new AvlNode{x, nullptr, nullptr};
        }
        else if (x.GetId() < this->element.GetId())
        {
            insert(x, t->left);
        }
        else if (this->element.GetId() < x.GetId())
        {
            insert(x, t->right);
        }
        else
        {
            return;
        }

        balance(t);
    }

    void remove(Farmer &x, AvlNode *&t)
    {
        if (t == nullptr)
            return;
        if (x.GetId() < this->element.GetId())
            remove(x, t->left);
        else if (this->element.GetId() < x.GetId())
            remove(x, t->right);
        else if (t->left != nullptr && t->right != nullptr) // Two children
        {
            this->element = findMin(t->right)->element;
```

```
void remove(Farmer &x, AvlNode *&t)
{
    if (t == nullptr)
        return;
    if (x.GetId() < this->element.GetId())
        remove(x, t->left);
    else if (this->element.GetId() < x.GetId())
        remove(x, t->right);
    else if (t->left != nullptr && t->right != nullptr) // Two children
    {
        this->element = findMin(t->right)->element;
        remove(this->element, t->right);
    }
    else
    {
        AvlNode *oldNode = t;
        t = (t->left != nullptr) ? t->left : t->right;
        delete oldNode;
    }

    balance(t);
}

bool contains(Farmer &x, AvlNode *t)
{
    if (t == nullptr)
        return false;
    else if (x.GetId() < this->element.GetId())
        return contains(x, t->left);
    else if (this->element.GetId() < x.GetId())
        return contains(x, t->right);
    else
        return true;
}

void makeEmpty(AvlNode *&t)
{
    if (t != nullptr)
    {
        makeEmpty(t->left);
        makeEmpty(t->right);
        delete t;
    }
    t = nullptr;
}

void printTree(AvlNode *t, ostream &out) const
{
    if (t != nullptr)
    {
        printTree(t->left, out);
        out << t->element.GetId();
        out << endl;
        printTree(t->right, out);
    }
}
```

```
        void balance(AvlNode *&t)
        {
            if (t == nullptr)
                return;

            if (GetHeight(t->left) - GetHeight(t->right) > ALLOWED_IMBALANCE)
            {
                if (GetHeight(t->left->left) >= GetHeight(t->left->right))
                    rotateWithLeftChild(t);
                else
                    doubleWithLeftChild(t);
            }
            else if (GetHeight(t->right) - GetHeight(t->left) > ALLOWED_IMBALANCE)
            {
                if (GetHeight(t->right->right) >= GetHeight(t->right->left))
                    rotateWithRightChild(t);
                else
                    doubleWithRightChild(t);
            }

            t->height = std::max(GetHeight(t->left), GetHeight(t->right)) + 1;
        }
    };
    AvlNode *GetRoot()
    {
        GetRootNode();
    }

    // function to print id and name of farmer of a specific area
    void PrintFarmerByArea(string areaName, int year, int month)
    {
        printFarmersArea(this->GetRootNode(), areaName, year, month);
    }

    // function to print id and name of farmer of a specific city
    void PrintFarmerByCity(string cityName, int year, int month)
    {
        printFarmersCity(this->GetRootNode(), cityName, year, month);
    }

    // function to print id and name of farmer of a specific wilaya
    void PrintFarmerByWilaya(string wilayaName, int year, int month)
    {
        printFarmersWilaya(this->GetRootNode(), wilayaName, year, month);
    }

private:
    AvlNode *root;

    AvlNode *GetRootNode()
    {
        return root;
    }

    // Private member functions for AvlTree
    void insert(Farmer &x, AvlNode *&t)
    {
```

```
private:
    AvlNode *root;

    AvlNode *GetRootNode()
    {
        return root;
    }

    // Private member functions for AvlTree
    void insert(Farmer &x, AvlNode *&t)
    {
        if (t == nullptr)
            t = new AvlNode{x, nullptr, nullptr};
        else if (x.GetId() < t->element.GetId())
            insert(x, t->left);
        else if (t->element.GetId() < x.GetId())
            insert(x, t->right);
        else
            return;

        balance(t);
    }

    void remove(Farmer &x, AvlNode *&t)
    {
        if (t == nullptr)
            return;
        if (x.GetId() < t->element.GetId())
            remove(x, t->left);
        else if (t->element.GetId() < x.GetId())
            remove(x, t->right);
        else if (t->left != nullptr && t->right != nullptr) // Two children
        {
            t->element = findMin(t->right)->element;
            remove(t->element, t->right);
        }
        else
        {
            AvlNode *oldNode = t;
            t = (t->left != nullptr) ? t->left : t->right;
            delete oldNode;
        }

        balance(t);
    }

    bool contains(Farmer &x, AvlNode *t) const
    {
        if (t == nullptr)
            return false;
        else if (x.GetId() < t->element.GetId())
            return contains(x, t->left);
        else if (t->element.GetId() < x.GetId())
            return contains(x, t->right);
        else
            return true;
    }
```

```
void makeEmpty(AvlNode *&t)
{
    if (t != nullptr)
    {
        makeEmpty(t->left);
        makeEmpty(t->right);
        delete t;
    }
    t = nullptr;
}

void printTree(AvlNode *t, std::ostream &out) const
{
    if (t != nullptr)
    {
        printTree(t->left, out);
        out << t->element.GetId();
        out << endl;
        printTree(t->right, out);
    }
}

AvlNode *clone(AvlNode *t) const
{
    if (t == nullptr)
        return nullptr;
    else
        return new AvlNode{t->element, clone(t->left), clone(t->right), t->height};
}

AvlNode *findMin(AvlNode *t) const
{
    if (t == nullptr)
        return nullptr;
    while (t->left != nullptr)
        t = t->left;
    return t;
}

void balance(AvlNode *&t)
{
    if (t == nullptr)
        return;

    if (height(t->left) - height(t->right) > ALLOWED_IMBALANCE)
    {
        if (height(t->left->left) >= height(t->left->right))
            rotateWithLeftChild(t);
        else
            doubleWithLeftChild(t);
    }
    else if (height(t->right) - height(t->left) > ALLOWED_IMBALANCE)
    {
        if (height(t->right->right) >= height(t->right->left))
            rotateWithRightChild(t);
        else
            doubleWithRightChild(t);
```

```
void balance(AvlNode *&t)
{
    if (t == nullptr)
        return;

    if (height(t->left) - height(t->right) > ALLOWED_IMBALANCE)
    {
        if (height(t->left->left) >= height(t->left->right))
            rotateWithLeftChild(t);
        else
            doubleWithLeftChild(t);
    }
    else if (height(t->right) - height(t->left) > ALLOWED_IMBALANCE)
    {
        if (height(t->right->right) >= height(t->right->left))
            rotateWithRightChild(t);
        else
            doubleWithRightChild(t);
    }

    t->height = std::max(height(t->left), height(t->right)) + 1;
}

int height(AvlNode *t) const
{
    return t == nullptr ? -1 : t->height;
}

void rotateWithLeftChild(AvlNode *&k2)
{
    AvlNode *k1 = k2->left;
    k2->left = k1->right;
    k1->right = k2;
    k2->height = std::max(height(k2->left), height(k2->right)) + 1;
    k1->height = std::max(height(k1->left), k2->height) + 1;
    k2 = k1;
}

void rotateWithRightChild(AvlNode *&k2)
{
    AvlNode *k1 = k2->right;
    k2->right = k1->left;
    k1->left = k2;
    k2->height = std::max(height(k2->left), height(k2->right)) + 1;
    k1->height = std::max(height(k1->right), k2->height) + 1;
    k2 = k1;
}

void doubleWithLeftChild(AvlNode *&k3)
{
    rotateWithRightChild(k3->left);
    rotateWithLeftChild(k3);
}

void doubleWithRightChild(AvlNode *&k1)
{
    rotateWithLeftChild(k1->right);
    rotateWithRightChild(k1);
```

```
    void doubleWithRightChild(AvlNode *&k1)
    {
        rotateWithLeftChild(k1->right);
        rotateWithRightChild(k1);
    }

    static const int ALLOWED_IMBALANCE = 1;

    // utility function to print all farmers in a specific area
    // handles area repitition by precising which city we are in
    void printFarmersArea(AvlNode *t, string areaName, int year, int month)
    {
        if (t != nullptr)
        {
            printFarmersArea(t->left, areaName, year, month);
            if (t->element.GetArea() == areaName && t->element.GetMonthlyStat().GetMonth() == month && t->element.GetMonthlyStat().GetYear() == year)
            {
                cout << "Farmer " << t->element.GetId() << ": " << t->element.GetName() << ", " << t->element.GetArea() << ", " << t->element.GetCity() << endl;
            }
            printFarmersArea(t->right, areaName, year, month);
        }
    }

    // utility function to print all farmers in a specific city
    // handles area repitition by precising which wilaya we are in
    void printFarmersCity(AvlNode *t, string cityName, int year, int month)
    {
        if (t != nullptr)
        {
            printFarmersCity(t->left, cityName, year, month);
            if (t->element.GetCity() == cityName && t->element.GetMonthlyStat().GetMonth() == month && t->element.GetMonthlyStat().GetYear() == year)
            {
                cout << "Farmer " << t->element.GetId() << ": " << t->element.GetName() << ", " << t->element.GetCity() << ", " << t->element.GetWilaya() << endl;
            }
            printFarmersCity(t->right, cityName, year, month);
        }
    }

    void printFarmersWilaya(AvlNode *t, string wilayaName, int year, int month)
    {
        if (t != nullptr)
        {
            printFarmersWilaya(t->left, wilayaName, year, month);
            if (t->element.GetWilaya() == wilayaName && t->element.GetMonthlyStat().GetMonth() == month && t->element.GetMonthlyStat().GetYear() == year)
            {
                cout << "Farmer " << t->element.GetId() << ": " << t->element.GetName() << ", " << t->element.GetWilaya() << endl;
            }
            printFarmersWilaya(t->right, wilayaName, year, month);
        }
    }
};

extern AvlTree *Cereals;
extern AvlTree *Vegetables;
extern AvlTree *Fruits;
extern AvlTree *Dates;
extern AvlTree *Olives;
```

AREA:

```cpp
#ifndef AREA_HPP
#define AREA_HPP
#include <string>
#include <vector>
#include "BinarySearchTree.hpp"

Comment Code
class Area
{

public:
    Area(std::string = "");          // constryctor
    std::string getnameArea() const; // geter;
    int expertReport(Farmer f);
    double salesAfterPenalities(Farmer);
    Farmer *findWinnerArea(BinarySearchTree *product, string areaName);
    Farmer *findWinner(BinarySearchTree::BinaryNode *root, const string &areaName);
    void PrintPinalitiesFarmersArea(BinarySearchTree *product, string areaName);


    bool isAreaNameExists(const std::string &name) const;
    void setnameArea(std::string name);

private:
    std::string nameArea;
    Farmer winner;
    void PrintPesticides(BinarySearchTree::BinaryNode *root, string areaName);

    void findWinnerHelper(BinarySearchTree::BinaryNode *root, const string &areaName, std::stack<Farmer> &winnerStack);
};

#endif // AREA_HPP

```

City :

```cpp
City.hpp > ...
Click here to ask Blackbox to help you code faster
#pragma once
#include <iostream>
#include <vector>
#include "Area.hpp"

class City
{
public:
    City(std::string = "");
    std::string getnameCity();
    void printFarmersCity();
    void PrintPinalitiesFarmersCity(string cityName, BinarySearchTree *product);
    Farmer *findWinnerCity(const std::string &cityName, BinarySearchTree *product);
    Area GetArea()
    {
        return area;
    }

    void setnameCity(std::string name);
    bool isCityNameExists(const std::string &name) const;
    std::vector<std::string> FillInVectorSameArea(const std::string &cityName);

private:
    std::string CityName;
    Area area;
    std::vector<Area> Areas;
    Farmer winnerCity;
    BinarySearchTree *Product;
    void PrintPesticides(string CityName, BinarySearchTree *Product);
    Farmer *findWinner(vector<string> &areas, const std::string &CityName, BinarySearchTree *product);
};

```

Wilaya:

```
Wilaya.hpp > ...
Click here to ask Blackbox to help you code faster
#pragma once
#include <iostream>
#include "City.hpp"

class Wilaya
{
public:
    Wilaya(std::string = "");
    std::string getnameWilaya();
    void PrintPesticides(string wilayaName, BinarySearchTree *product);
    Farmer *findWinnerWilaya(const std::string &WilayaName, BinarySearchTree *product);
    Area getArea() { return area; }
    void setnameWilaya(std::string name);
    bool isWilayaNameExists(const std::string &name) const;
    std::vector<std::string> FillInVectorSameCity(const std::string &WilayaName);
    void PrintPinalitiesFarmersWilaya(string wilayaName, BinarySearchTree *product);

private:
    Area area;
    std::string WilayaName;
    std::vector<City> cities;
    Farmer *winnerWilaya;
};

```

Country:

```
country.hpp > ...
Click here to ask Blackbox to help you code faster
#pragma once
#include <iostream>
#include "Wilaya.hpp"

class Country
{
public:
    Country(){};
    void printFarmer(BinarySearchTree *Product); // call the print in class farmer
    void PrintPinalitiesFarmersCountry(BinarySearchTree *Product);
    std::vector<std::string> FillInVector();
    Farmer *findWinnerCountry(BinarySearchTree *product);


private:
    std::string CountryName;
    Area area;
    std::vector<Wilaya> Wilayas;
    Farmer *winnerCountry;
};

```

Farmer:

Farmer.hpp > ...

Click here to ask Blackbox to help you code faster

```cpp
#pragma once
#include <iostream>
#include <fstream>
#include <sstream>
#include <algorithm>
#include "MonthlyReport.hpp"
using namespace std;

// Farmer class
class Farmer
{
private:
    int ID;
    string name;
    int age;
    string wilaya;
    string city;
    string area;
    string gender;
    int LandId;

    int counter = 0;
    MonthlyReport MonthlyStats;

public:
    // FarmerID,LastName,Age,Gender,LandID,Area,City,Wilaya
    Farmer(int farmer_id, string L_name, int ag, string gen, int L_ID, string wi, string ci, string ar)
    {
        ID = farmer_id;
        name = L_name;
        age = ag;
        gender = gen;
        LandId = L_ID;
        wilaya = wi;
        city = ci;
        area = ar;
    };

    string GetArea()
    {
        return area;
    }
    string GetWilaya()
    {
        return wilaya;
    }
    string GetCity()
```

Monthly report:

MonthlyReport.hpp > MonthlyReport > SetProductSales()

Click here to ask Blackbox to help you code faster

```
#pragma once
#include <iostream>
#include <algorithm>
#include "Date.hpp"
using namespace std;

// monthly report class where monthly statistics are kept
class MonthlyReport
{
private:
    Date report_date;
    int Farmer_ID;
    double Sales;
    string productName[5];
    int ProductNum;
    int helpers;
    double WaterConsumption;
    double ElectricityConsumption;
    int pesticide;
    int ProducesFruits;
    int ProducesVegetables;
    int ProducesCereals;
    int ProducesDates;
    int ProducesOlives;
    double SalesFruits;
    double SalesVegetables;
    double SalesCereals;
    double SalesDates;
    double SalesOlives;

public:
    MonthlyReport(){};
    MonthlyReport(int id, int numhelp, int PF, int PV, int PC, int PD, int PO, double SF, double SV, double SC, double SD, double SO, double water, double elec, int mo, int yea)
    {
        Farmer_ID = id;
        helpers = numhelp;
        ProducesFruits = PF;
        ProducesVegetables = PV;
        ProducesCereals = PC;
        ProducesDates = PD;
        ProducesOlives = PO;
        SalesFruits = SF;
        SalesVegetables = SV;
        SalesCereals = SC;
        SalesDates = SD;
        SalesOlives = SO;
```